Vídeo 4.4: Conectando a API ao SQLite e testando a persistência

Duração-alvo: 7 min | **Conceitos:** conexão com o banco, consultas com ? e persistência | **O aluno sai sabendo:** guardar os livros em um banco SQLite e conferir que eles sobrevivem ao reinício.

Antes de gravar: partir do código do fim do módulo 3 (versão v3), sem o arquivo biblioteca.db na pasta. O banco começa vazio: os livros serão cadastrados com POST durante o vídeo. Este é o vídeo mais denso do módulo: gravar a troca das cinco rotas em ritmo acelerado e narrar o padrão, em vez de cada linha. Ao fim, marcar o código como versão v4.

[0:00] Abertura

Na tela: título do vídeo.

Fala: “Agora vamos juntar tudo: trocar a lista que vive na memória por um banco SQLite que guarda os livros no disco. No final, você vai reiniciar o servidor e ver os livros continuando lá.”

[0:30] Instalando

Na tela: no terminal, digitar o comando abaixo.

npm install sqlite3 sqlite
Fala: “Precisamos de dois pacotes. O sqlite3 é o motor que conversa com o banco SQLite. O sqlite é um ajudante que deixa esse motor trabalhar com async e await, do jeito que acabamos de aprender.”

[1:15] Abrindo o banco e criando a tabela

Na tela: no server.js, apagar a lista de livros e o contador, e escrever a função abaixo.

```
const sqlite3 = require('sqlite3');
 const { open } = require('sqlite');

 async function abrirBanco() {
   const db = await open({
         filename: 'biblioteca.db',
         driver: sqlite3.Database
   });

   await db.exec(`
         CREATE TABLE IF NOT EXISTS livros (
         id INTEGER PRIMARY KEY AUTOINCREMENT,
         titulo TEXT NOT NULL,
         autor TEXT NOT NULL
         )
   `);
```

```
  return db;
}
```
Fala: “Esta função abre o banco. O filename é o nome do arquivo onde os dados ficam: biblioteca ponto db. Se o arquivo não existe, o SQLite cria. Depois mandamos um comando SQL para criar a tabela livros, mas só se ela ainda não existir. É o IF NOT EXISTS, e por isso podemos ligar o servidor mil vezes sem apagar nada. A tabela tem três colunas: id, que é a chave primária e se numera sozinha; título e autor, que são textos e não podem ficar vazios. Reparem no await antes de cada passo: abrir o banco e criar a tabela demoram. É o depósito.”

[2:45] Ligando o servidor depois do banco

Na tela: trocar o app.listen pela função iniciar.

```
let db;

async function iniciar() {
  db = await abrirBanco();

  app.listen(3000, () => {
        console.log('Servidor rodando em http://localhost:3000');
  });
}

iniciar();
```
Fala: “Troco o app ponto listen por uma função chamada iniciar. Ela primeiro abre o banco e só então abre o balcão para atender. Assim, nenhum pedido chega antes de o banco estar pronto.”

[3:30] Trocando as rotas

Na tela: reescrever as rotas de livros, uma de cada vez, em ritmo acelerado. A rota /status e o express.json() continuam como estavam.

```
app.get('/livros', async (req, res) => {
  const { autor } = req.query;

  if (autor) {
        const filtrados = await db.all(
        'SELECT * FROM livros WHERE autor LIKE ?',
        `%${autor}%`
        );
        return res.json(filtrados);
  }

  const livros = await db.all('SELECT * FROM livros');
  res.json(livros);
});

app.get('/livros/:id', async (req, res) => {
```

```
  const livro = await db.get('SELECT * FROM livros WHERE id = ?', req.params.id);

  if (!livro) {
        return res.status(404).json({ erro: 'Livro não encontrado' });
  }

  res.json(livro);
});

app.post('/livros', async (req, res) => {
  const { titulo, autor } = req.body;
  const resultado = await db.run(
        'INSERT INTO livros (titulo, autor) VALUES (?, ?)',
        titulo, autor
  );

  res.status(201).json({ id: resultado.lastID, titulo, autor });
});

app.put('/livros/:id', async (req, res) => {
  const { titulo, autor } = req.body;
  const resultado = await db.run(
        'UPDATE livros SET titulo = ?, autor = ? WHERE id = ?',
        titulo, autor, req.params.id
  );

  if (resultado.changes === 0) {
        return res.status(404).json({ erro: 'Livro não encontrado' });
  }

  res.json({ id: Number(req.params.id), titulo, autor });
});

app.delete('/livros/:id', async (req, res) => {
  const resultado = await db.run('DELETE FROM livros WHERE id = ?', req.params.id);

  if (resultado.changes === 0) {
        return res.status(404).json({ erro: 'Livro não encontrado' });
  }

  res.status(204).send();
});
```

Fala: “Agora cada rota. O padrão é sempre o mesmo: a função ganha async, e cada chamada ao banco ganha await. O db ponto all devolve todas as linhas que combinam. O db ponto get devolve uma linha só. E o db ponto run executa um comando que altera dados e informa o que aconteceu: o número do livro criado, em lastID, e quantos registros foram alterados, em changes. Nos comandos SQL aparece um ponto de interrogação no lugar de cada valor, e os valores vêm depois, separados. É um cuidado de segurança: nunca

juntamos o texto que o cliente mandou dentro do comando SQL. Quem coloca os valores nos lugares é o banco, e isso protege contra um tipo de ataque chamado injeção de SQL, em que alguém envia um texto malicioso para tentar enganar o banco. No PUT e no DELETE, se changes for zero, nenhum livro foi encontrado, e respondemos 404."

[5:45] Testando a persistência

Na tela: rodar node server.js; cadastrar dois livros com POST; listar; parar o servidor, ligar de novo e listar outra vez. Mostrar o arquivo biblioteca.db que apareceu na pasta.

Fala: "Vamos testar o que importa. Rodo o servidor. Na primeira vez o banco está vazio, então cadastro dois livros com POST. Listo: estão lá. Agora o teste de fogo: paro o servidor e ligo de novo. Listo outra vez... Os livros continuam aqui! Repare no arquivo biblioteca ponto db que apareceu na pasta: é ali que o SQLite guarda tudo. Os dados agora estão no fichário, não na lousa."

[6:40] Resumo

Na tela: três frases curtas.

Fala: "Resumindo: abrimos um banco SQLite, criamos a tabela de livros e trocamos a lista da memória por comandos SQL, sempre com async e await. Os dados agora sobrevivem ao reinício. Um aviso: o arquivo biblioteca ponto db guarda os dados reais e não deve ir para o repositório público. Falaremos disso no último módulo. No próximo módulo vamos proteger a API contra dados ruins."

Exercício sugerido: cadastrar três livros, reiniciar o servidor e conferir que eles continuam na lista; depois excluir um e reiniciar de novo.